NEOBRIM
Elevating Homes

Software Requirements Specifications (SRS) for Fan Class

Requirement #2: Fan Class Module

- **Assignee:** DhanushKumar R.
- **Role:** Deep Learning (AI) Intern
- **Date of Assignment:** January 27, 2024
- **Deadline:** February 3, 2024

Home Automation System Implementation

Objective: Design and implement a comprehensive home automation system with intelligent control mechanisms using Python classes. The system should include classes for managing different types of lights, an aggregator class to coordinate various components, and an AI agent class for predicting and optimizing the state of lights.

1. **Fan Class:**

   Objective: Create a **Fan** class to represent a fan in the home automation system. The class should offer functionalities to control fan speed and fetch the current status.

   Specifications:

   - Properties:
     - **Fan_Speed**: Integer from 0 to 10, representing the number of speed levels (0 for OFF).
     - **location**: String specifying the location of the fan.
   - Methods:
     - **__init__(self, Fan_Speed: int, location: str, *args, **kwargs)**: Initialize the fan object with given parameters.
     - **control_fan(self, speed: int)**: Control the fan speed by sending the relevant speed.
     - **fetch_status(self) -> dict**: Fetch the current status of the fan.

2. **Aggregator Class:**

   Objective: Develop an **Aggregator** class to act as the central controller for the home automation system, specifically managing fan objects. This class should coordinate interactions, handle AI predictions, and maintain a CSV log.

   Specifications:

   - Properties:

- **temperature**: Current temperature in the environment.
- **time**: Current time.
- **motion**: Motion status in the area.
- **AC_temp_status**: Status of the AC temperature.
- Other relevant parameters.

- Methods:
  - **__init__(self, temperature, time, motion, AC_temp_status, *args, **kwargs)**: Initialize the Aggregator object with given parameters.
  - **define_fan_objects(self, fan_info: List[Dict[str, Union[int, str]]])**: Define objects of the **Fan** class based on provided information, including the number of speed levels.
  - **define_ai_object(self, ai_params: dict) -> AIAgent**: Define an object of the **AIAgent** class with given parameters.
  - **invoke_fan_control(self, fan_obj: Fan)**: Invoke the relevant control function of the **Fan** class on the provided object.
  - **save_ai_output_to_csv(self, ai_output: str)**: Save the AI output in a CSV file (for the last 15 days).
  - **compute_difference(self, previous_state: dict, current_state: dict) -> bool**: Compute the difference between previous and current states. Set manual_override boolean to true if the difference is not null and provide feedback to the AI model based on the degree of difference.
  - **compute_future_difference(self, current_state: dict, next_predicted_state: dict) -> bool**: Compute the difference between current and predicted states. If the difference is None, do not send a signal.
  - **send_signal(self)**: Empty function to send a signal.

3. **AI Agent Class:**

   Objective: Implement an **AI Agent** class serving as an intelligent model predicting the state of fans. The class should support both Deep Learning and Reinforcement Learning approaches. For Deep Learning, a dummy dataset must be created, and for Reinforcement Learning, a reward function needs to be defined. The AI agent should only turn on a fan if the confidence percentage for ON is more than 85%.

   Specifications:

   - Properties:
     - The properties depend on whether it's a Deep Learning or Reinforcement Learning model.
   - Methods:

- **__init__(self, model_type: str, *args, **kwargs)**: Initialize the AIAgent object with the specified model type.
- If it's a Deep Learning model:
    - **train(self, dataset: List[Dict[str, Any]])**: Train the model using the provided dataset.
    - **predict(self, input_data: Any) -> Tuple[int, float]**: Predict the state and confidence based on the input data.
- If it's a Reinforcement Learning model:
    - **define_reward_function(self, *args, **kwargs)**: Define the reward function for the RL model.
    - **train_rl(self, *args, **kwargs)**: Train the RL model.
    - **act(self, *args, **kwargs) -> str**: Take an action based on the current state.

Note: Certain basic assumptions are made, and flexibility is allowed for further improvement. Modify the classes and methods as needed to enhance the functionality and adapt to specific requirements.